

Spécifications Techniques

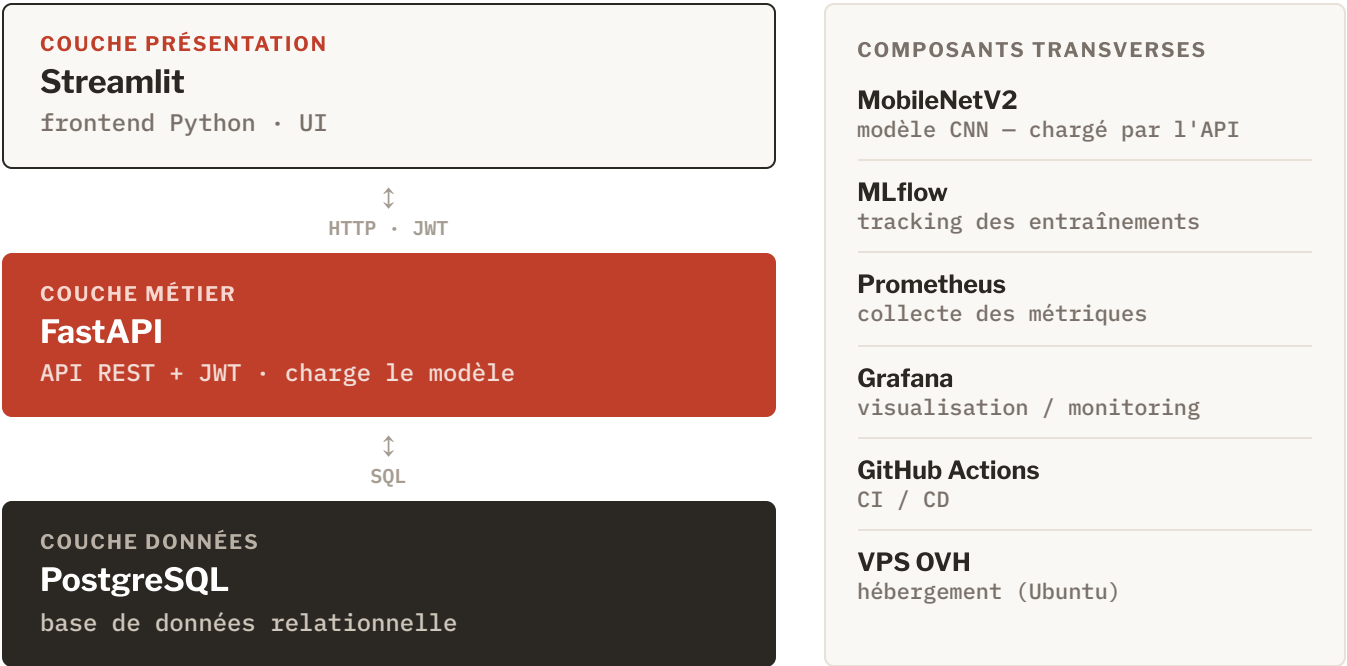
Architecture, choix technologiques, flux de données et environnement d'exécution.

AUTEUR	ÉPREUVE	COMPÉTENCES	ANNÉE
Sullivan Moreau	E4 — Conception & réalisation	C14 → C19	2026

LIVRABLE 1

Architecture n-tiers

Architecture en 3 couches, avec les composants transverses chargés ou orchestrés autour de la couche métier.



LIVRABLE 2

Choix des langages et outils

Chaque brique technique est justifiée au regard du contexte : prototype ML, performance, maturité et démarche éco-responsable.

COMPOSANT	TECHNOLOGIE	JUSTIFICATION
Frontend	Streamlit	Python natif, pas de JavaScript à apprendre, idéal pour un prototype ML.
API	FastAPI	Performance asynchrone, validation automatique Pydantic, OpenAPI intégré.
Modèle	MobileNetV2 / TensorFlow	Léger, performant sur CPU, transfer learning efficace.
BDD	PostgreSQL	Relationnel, mature, RGPD-friendly, open source.
Tracking	MLflow	Standard MLOps, log des métriques et modèles, interface web incluse.
Monitoring	Prometheus + Grafana	Stack standard DevOps, open source, éco-responsable.
CI / CD	GitHub Actions	Intégré au repo, gratuit pour les projets publics, standard industrie.
Conteneurisation	Docker	Reproductibilité, déploiement simplifié.
Gestionnaire paquets	uv	Plus rapide que pip, standard moderne Python.
Hébergement	VPS OVH	Souveraineté des données (hébergeur français / européen), RGPD.

LIVRABLE 3

Flux de données et zones de stockage

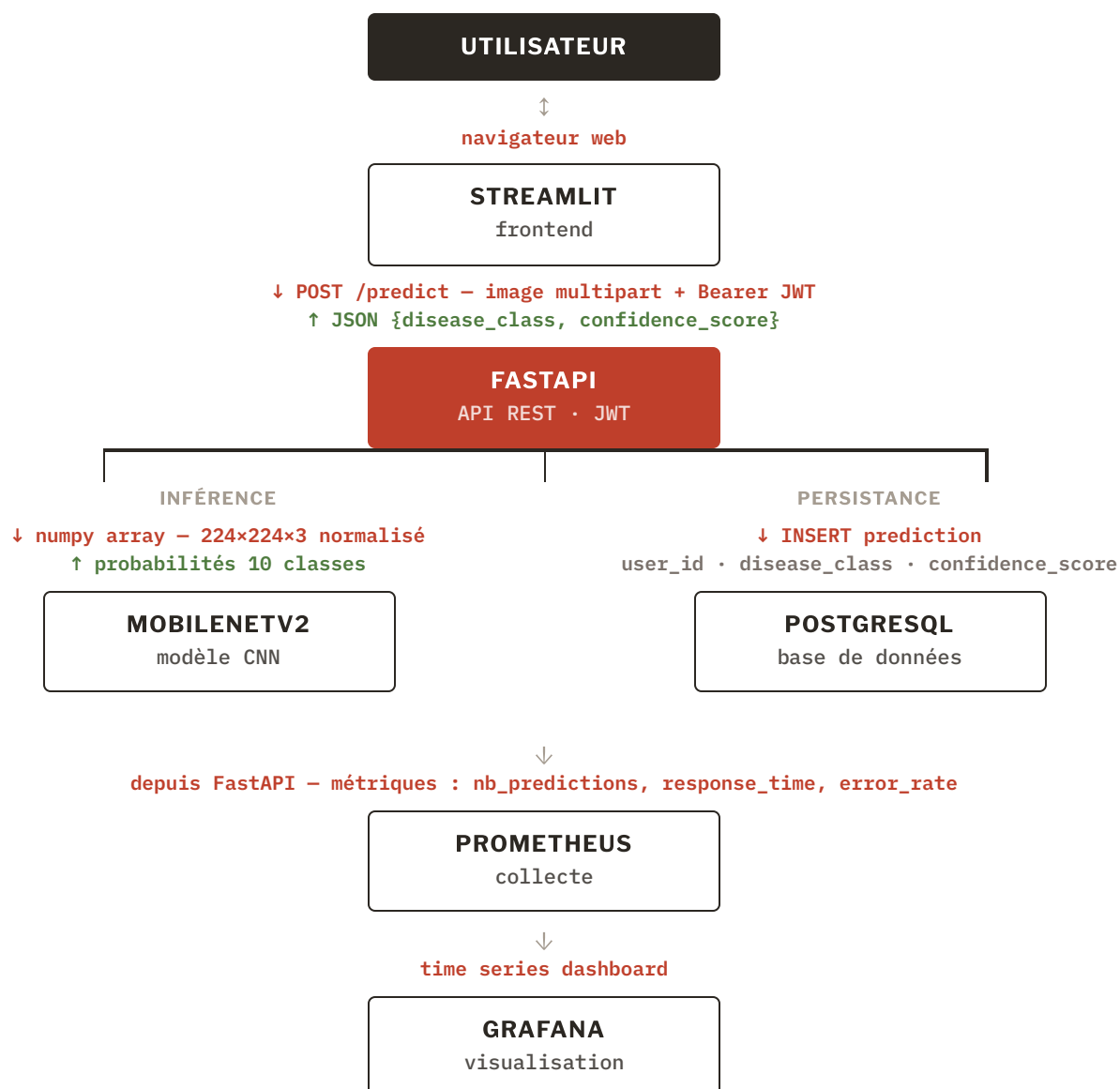
Localisation et nature de chaque donnée manipulée par l'application.

DONNÉE	ZONE DE STOCKAGE	DÉTAIL
Images uploadées	NON STOCKÉES	Traitement en mémoire uniquement — conformité RGPD.
Prédictions	POSTGRESQL	id, user_id, disease_class, confidence_score, created_at
Utilisateurs	POSTGRESQL	id, email, password_hash, role, created_at
Modèle entraîné	FICHIER .KERAS	Sur le VPS — chargé au démarrage de l'API.
Métriques MLflow	DOSSIER LOCAL	mlruns/
Métriques Prometheus	MÉMOIRE	Séries temporelles (time series) en mémoire Prometheus.
Logs application	LOGURU	Fichiers locaux sur le VPS.

LIVRABLE 4

Diagramme de flux de données détaillé

Mêmes composants que le document de modélisation (#26), avec les formats de données échangés précisés. ■ requête / écriture · ■ réponse.

**LIVRABLE 5****Justification éco-responsable**

Services éco-responsables favorisés conformément aux recommandations du référentiel Simplon.

- ◆ **Open source & gratuit** — choix de services open source plutôt que des SaaS propriétaires, évitant la surconsommation liée aux grandes plateformes cloud.

◆ **MobileNetV2** — modèle léger optimisé pour CPU, aucun GPU requis en inférence, donc moins de consommation énergétique.

◆ **uv** — gestionnaire de paquets ~10× plus rapide que pip, donc moins de temps CPU lors des installations.
- ◆ **VPS OVH** — hébergeur européen avec une démarche éco-responsable, datacenters certifiés ISO 50001.

◆ **Pas de stockage d'images** — réduction directe de l'empreinte de stockage.

◆ **Docker** — images légères basées sur python:3.11-slim, réduisant la taille des conteneurs.

LIVRABLE 6

Environnement d'exécution

ÉLÉMENT	VALEUR
Système d'exploitation	Ubuntu 22.04 – VPS OVH
Python	3.11 – verrouillé via <code>.python-version</code>
Gestionnaire de paquets	uv 0.4+
Conteneurisation	Docker + Docker Compose
Task runner	just
Branches	main (prod) · develop (dev)
CI / CD	GitHub Actions

DÉPENDANCES PRINCIPALES — VERSIONS MINIMALES

<code>fastapi</code>	<code>≥ 0.111.0</code>	<code>uvicorn</code>	<code>≥ 0.29.0</code>
<code>streamlit</code>	<code>≥ 1.35.0</code>	<code>tensorflow</code>	<code>≥ 2.16.1</code>
<code>mlflow</code>	<code>≥ 2.13.0</code>	<code>sqlalchemy</code>	<code>≥ 2.0.30</code>
<code>alembic</code>	<code>≥ 1.13.0</code>	<code>psycpg2-binary</code>	<code>≥ 2.9.9</code>
<code>prometheus-client</code>	<code>≥ 0.20.0</code>	<code>loguru</code>	<code>≥ 0.7.2</code>
<code>python-jose[cryptography]</code>			<code>≥ 3.3.0</code>

Spécifications techniques — diplôme DevIA Simplon — Sullivan Moreau — 2026